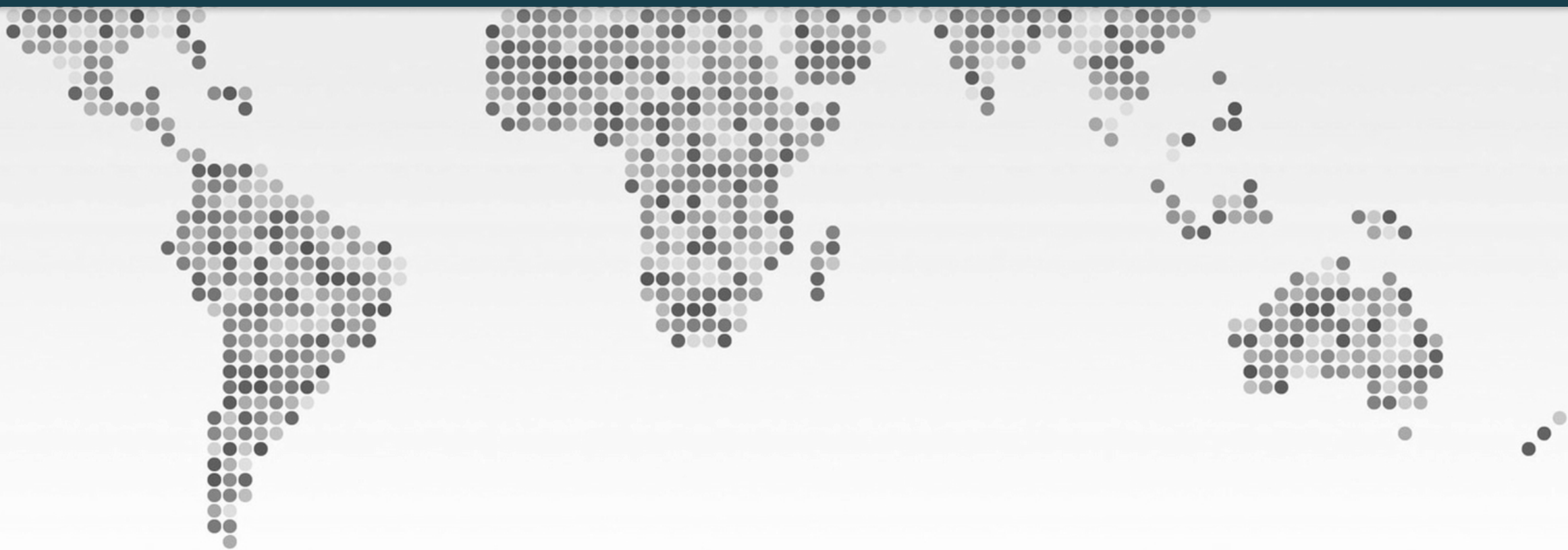


Software Engineering for Artificial Intelligence

Software Architectures and Deployment of AI-enabled Systems



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

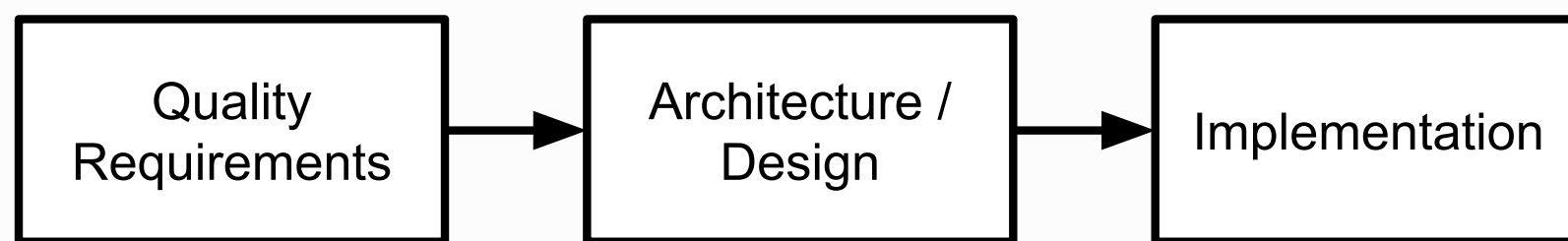
Thinking like an Architect

Right after the requirements analysis, programmers have a tendency to jump right into implementations, but this risks building systems that do not actually meet the users' needs, that do not achieve the desired qualities, that do not meet the business goals, and that are hard to change when needed.

Software architecture is the part of the design process that focuses on those early decisions that are most important for achieving the quality requirements of a system (e.g., scalability, flexibility).

Architectural design decisions are fundamental and hard to change later without fully redesigning the system. Yet, those decisions are often difficult to get right in the first place, requiring a good understanding of requirements, of consequences of different designs, and of the trade-offs involved.

The fundamental insight of software architecture is to seriously consider a system's quality requirements early on and design the system to support those qualities.



Architecture is the binding element that guides the implementation to meet the (quality) requirements of the system.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Every System is Different!

Ideally, we would like to describe a *common* architecture of integrating machine learning into a software system, but systems differ widely in their needs and quality requirements. To illustrate the point, let us just contrast four different examples:

1

A *personalized recommendation* component for a music streaming service needs to provide recommendations **fast** and **at scale** and needs to be flexible enough to allow frequent experimentation and automatic updates.

We can plausibly deploy the model inference service as a **microservice** in some massively parallel cloud infrastructure. **Pipeline automation and monitoring** will be important to foster regular updates and experiments.

2

A *transcription service* that uses speech-to-text models to transcribe audio files **does not need to provide real-time results**, but should be elastic to scale with changing demands.

Each user only occasionally interacts with the system in irregular intervals, uploading audio files when needed. **Inference** itself is **computationally expensive** and requires **specialized hardware**.

Cloud resources and a working queue are likely a reasonable fit to schedule and balance the work. At the same time, **model updates are not as frequent**, so human steps and manual testing in the update and deployment process may be acceptable.

Thinking like an Architect: Every System is Different!

Ideally, we would like to describe a *common* architecture of integrating machine learning into a software system, but systems differ widely in their needs and quality requirements. To illustrate the point, let us just contrast four different examples:

3

An *autonomous train* typically uses dozens of models that need to work on onboard computers in **real-time**. Locally available hardware sets **hard constraints** on what is computationally feasible.

Mistakes can be fatal, so non-ML components will provide significant logic to ensure safety, **interacting closely with the ML components**. Experimenting with different model versions in practice is limited by safety concerns.

4

A *privacy-focused smart keyboard* on a mobile phone may **continuously train** a model for autocompletion and autocorrection locally on the phone. Here, not only **inference**, but also **data processing and training happens locally** on a battery-powered device, possibly using novel federated machine-learning algorithms.

When collecting telemetry to monitor whether updates lead to better predictions, developers have to make careful decisions about what data to collect and share.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Every System is Different!

Ideally, we would like to describe a *common* architecture of integrating machine learning into a software system, but systems differ widely in their needs and quality requirements. To illustrate the point, let us just contrast four different examples.

These few examples already illustrate many kinds of differences in quality requirements that will inform many design decisions, including:

- Different degrees of importance of machine learning to the system's operation affecting reliability requirements;
- Different frequency, latency requirements, and cost of model inference requests;
- Different amount of data to process during model inference or model training;
- Different frequency of model updates;
- Different opportunities and requirements for collecting telemetry data and conducting experiments in production;
- Different levels of privacy and safety requirements based on different levels of risk.

As a consequence, these systems all face very different challenges and will explore different design decisions that serve their specific needs.

Thinking like an Architect: Common Architectural Challenges and System Structures

There are many design challenges in every system, and the specifics will be different from system to system. Yet, when designing an AI-enabled system, three are the most relevant point to tackle.

1

Divide and Conquer. A fundamental decision that needs to be made early in the development process is how to *decompose* the system, such that different teams can work on different components of the project. A simple approach would first split the model development from the system development, as different roles are in charge of their design (AI engineers and software engineers, respectively).

However, the various ML and non-ML components in a system interact, often in subtle ways. Clearly understanding interactions helps to identify where continued coordination is needed across teams.

A component's interface plays a crucial role in the collaboration between teams building a system as a negotiation and collaboration point. The interface is where teams (re-)negotiate how to divide work and how to assign responsibilities.

Again, remember that the artificial intelligence pipeline may interact with other components in intricate ways: **architecting means finding an optimal distribution of responsibilities that minimizes the risks of miscommunications** which, in turn, may lead to erroneous implementations and long-term socio-technical issues.

Thinking like an Architect: Common Architectural Challenges and System Structures

There are many design challenges in every system, and the specifics will be different from system to system. Yet, when designing an AI-enabled system, three are the most relevant point to tackle.

2

Facilitating experimentation and updates with confidence. Anticipating what parts of the system are more likely to change or what qualities will become more important in the future allows it to design the system such that those changes are easier when needed. In addition, designing the system such that it can be updated easily and frequently with confidence in the updates will lower barriers to change and increase confidence in deployments.

3

Operating at scale or with limited resources. Many ML-enabled systems operate with very large amounts of data and should be deployed at massive scale in the cloud. Distributed computing will be unavoidable as data and computations exceed the capabilities of even the fastest supercomputers.

In addition to the technical complexities and new failure points introduced with distributed computing, designers must now balance latency, throughput, and fault tolerance depending on the system's quality requirements.

At the same time, other systems may deploy AI on embedded devices with limited resources. This may require creative solutions to operate with limited resources and still provide opportunities for collecting telemetry and updating the system.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

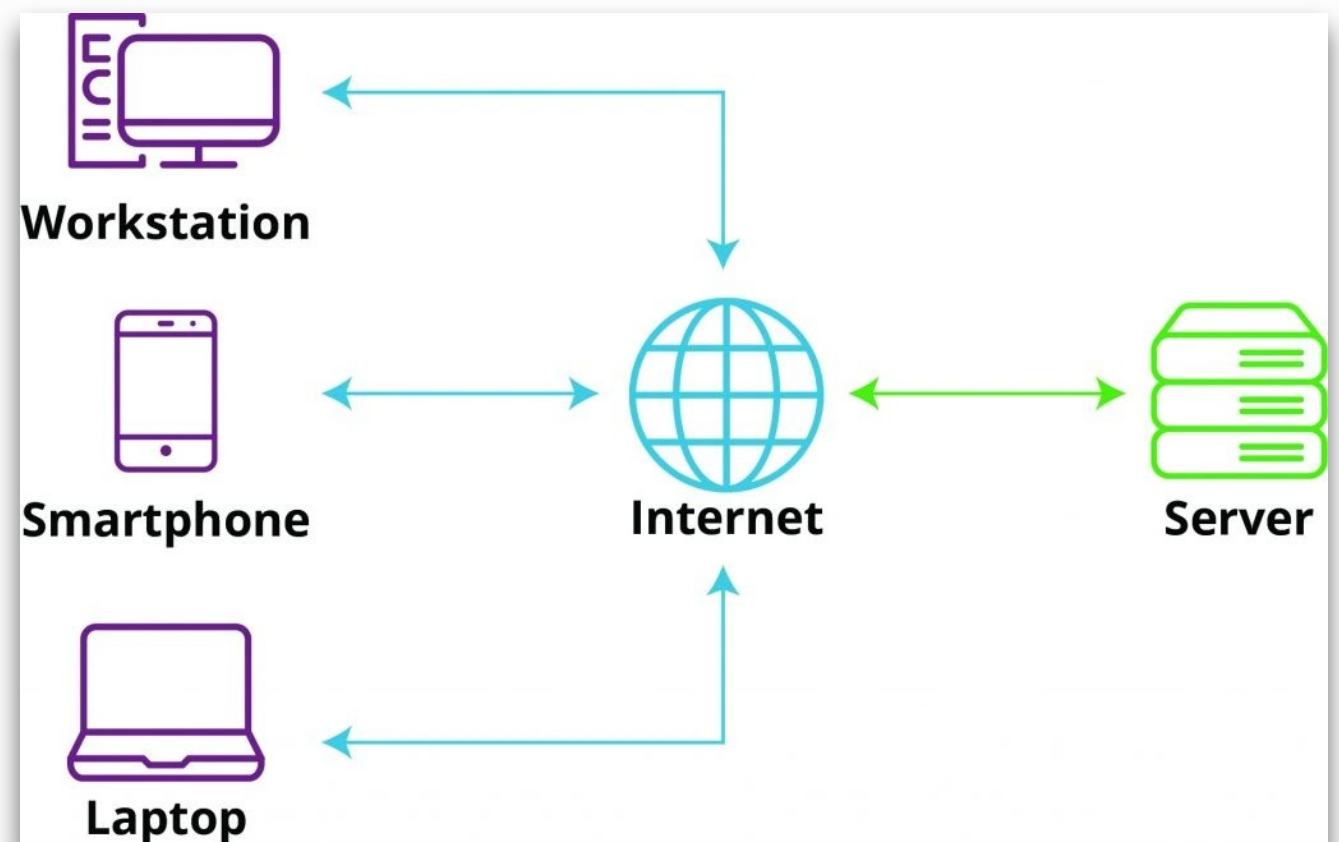
Thinking like an Architect: Common Architectural Challenges and System Structures

To accommodate the needs of an AI-enabled system, various, common architectural styles may be employed. Understanding such common system structures is useful for considerations of how to fit machine-learning components into a system.

1

Client-Server Architecture. Computations are split across multiple machines. A server provides functionality to multiple clients, typically over a network connection. This way, computational resources can be shared centrally for many users, whereas clients can remain relatively simple. The client invokes communication with the server.

Clients handle lightweight computations, UI interactions, input processing, and local caching.



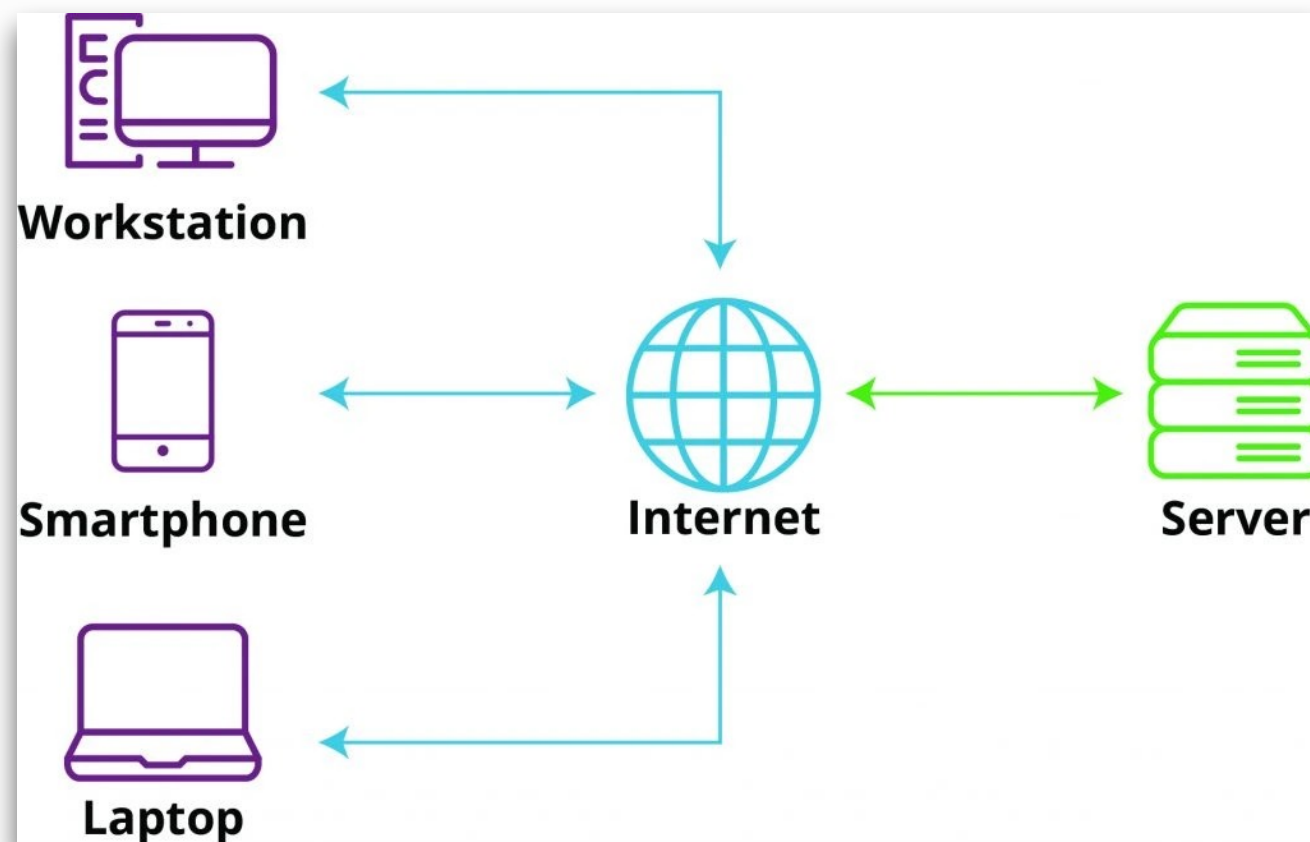
Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

To accommodate the needs of an AI-enabled system, various, common architectural styles may be employed. Understanding such common system structures is useful for considerations of how to fit machine-learning components into a system.

1

Client-Server Architecture. Computations are split across multiple machines. A server provides functionality to multiple clients, typically over a network connection. This way, computational resources can be shared centrally for many users, whereas clients can remain relatively simple. The client invokes communication with the server.



The server manages AI model inference, data processing, and storage, leveraging high-performance computing resources.

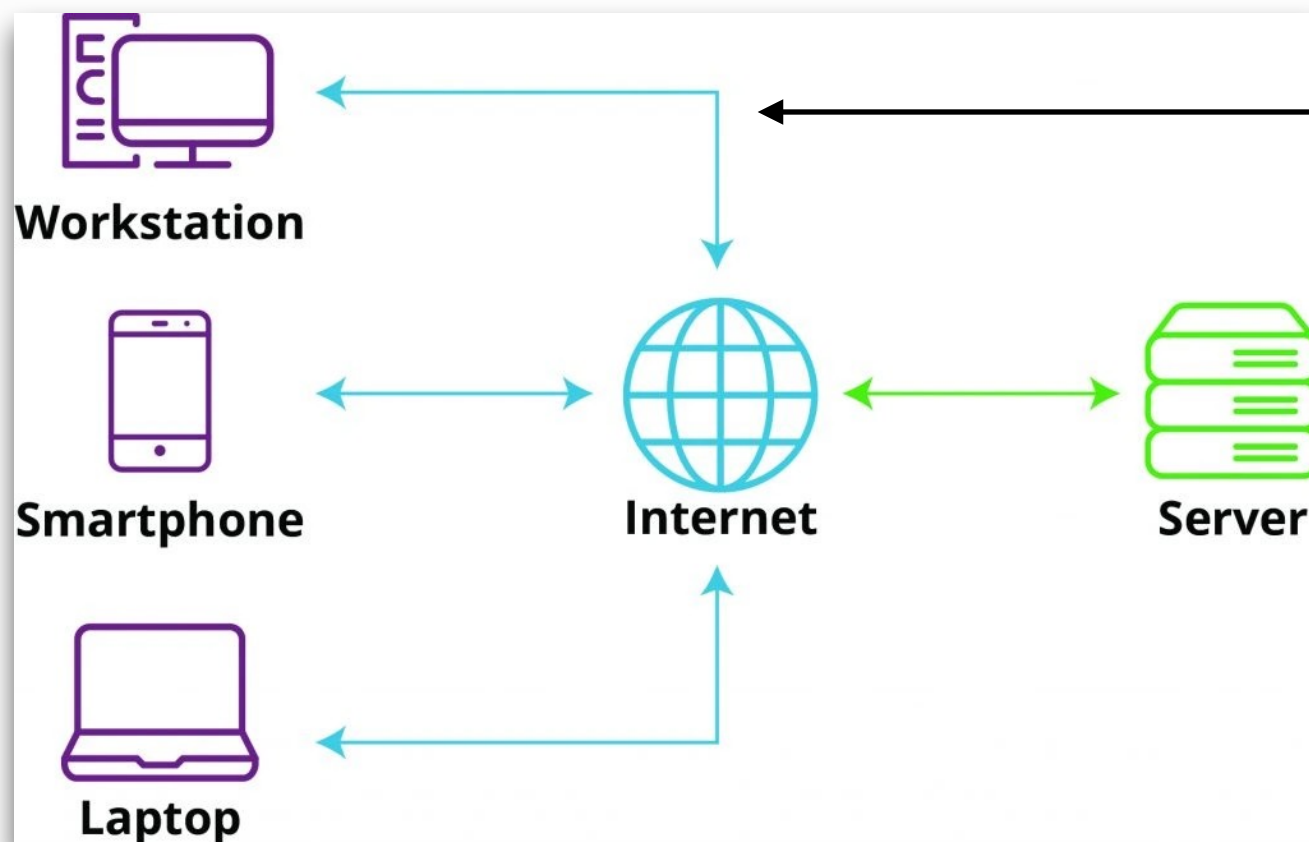
Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

To accommodate the needs of an AI-enabled system, various, common architectural styles may be employed. Understanding such common system structures is useful for considerations of how to fit machine-learning components into a system.

1

Client-Server Architecture. Computations are split across multiple machines. A server provides functionality to multiple clients, typically over a network connection. This way, computational resources can be shared centrally for many users, whereas clients can remain relatively simple. The client invokes communication with the server.



A key decision here is the communication protocol, i.e., how the servers actually serve clients:

RESTful APIs (HTTP/HTTPS): Simple but not ideal for real-time applications.

WebSocket: Suitable for interactive AI applications (e.g., chatbots, AI recommenders).

Message Queuing Telemetry Transport: Best for IoT-based AI applications needing low-bandwidth communication.

Thinking like an Architect: Common Architectural Challenges and System Structures

To accommodate the needs of an AI-enabled system, various, common architectural styles may be employed. Understanding such common system structures is useful for considerations of how to fit machine-learning components into a system.

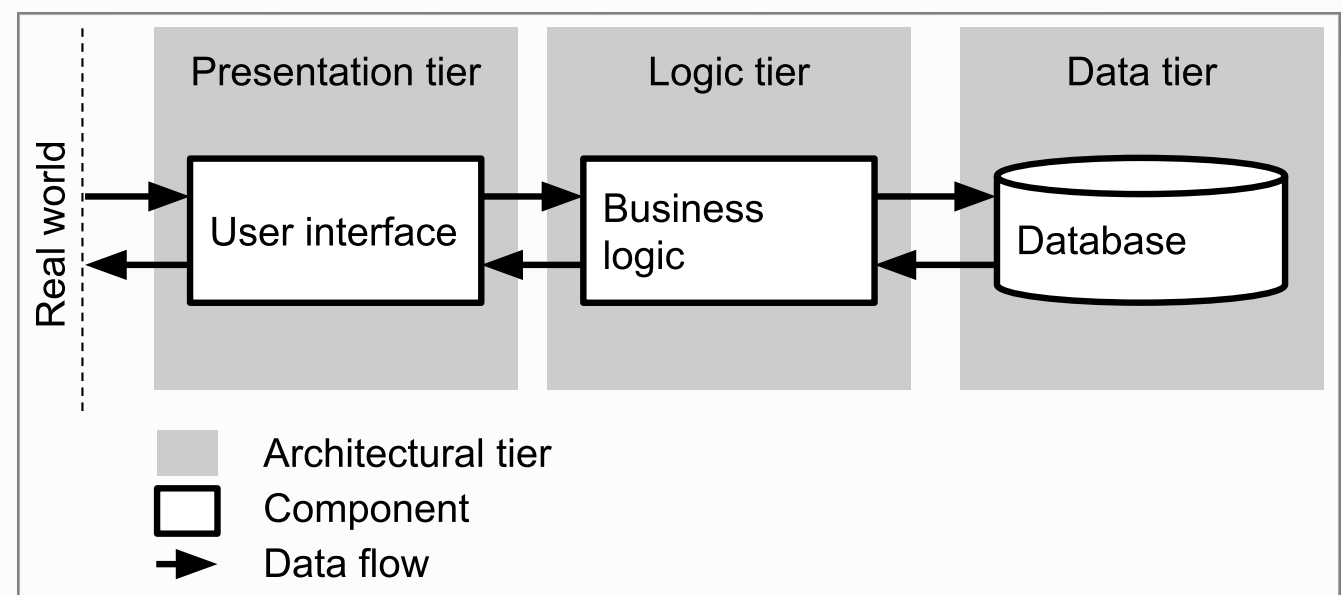
2

Multi-Tier Architecture. Computation and data storage is organized into multiple layers or tiers (on the same or different machines): Clients make requests to servers, which make requests to other servers, and so forth. Higher tiers send requests to lower tiers, but not the other way around.

The most common structure is a three-tier architecture with a presentation tier, a logic tier, and a data tier—the data tier manages data storage; the logic tier implements business logic such as processing transactions to change data; and the presentation tier provides the user interface through which clients interact with the system.

—————→

This structure is common for business and web applications and can be conceptually extended with components related to machine learning (more later in this lecture)



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

To accommodate the needs of an AI-enabled system, various, common architectural styles may be employed. Understanding such common system structures is useful for considerations of how to fit machine-learning components into a system.

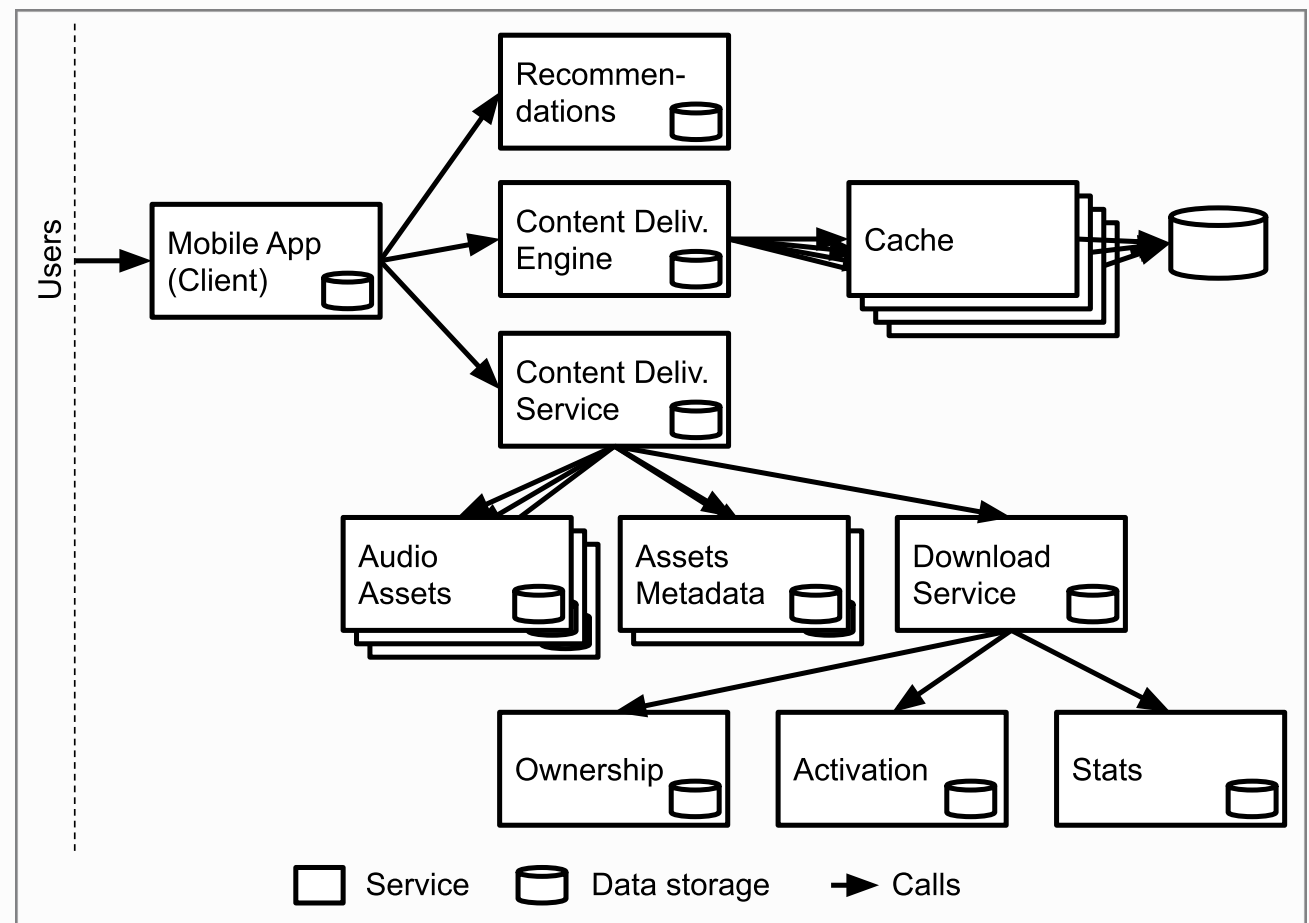
3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

→

This design allows independent deployment, versioning, and scaling of services and flexible routing of requests at the network level.

Many modern, scalable web-based systems use this design: let's further elaborate on this architecture!



Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Essentially, SOA is an architectural style where software components (services) communicate over a network via standardized protocols. It has been defined to emphasize reusability, loose coupling, and interoperability.

Service. *A self-contained, modular software component that performs a specific function and communicates with other services over a network.*

Each service provides a repeatable business activity (e.g., “Recommendation Service”). It may be composed of other auxiliary services.

Services communicate between them via an Enterprise Service Bus (ESB).

In SOA, a service may either be *stateful* or *stateless*:

A *stateful* service retains session or user-specific data across multiple requests. For instance, imagine a **chatbot session service** that remembers past user interactions.

As one can easily understand, a stateful service requires additional storage (e.g., session databases) and is, therefore, dependent on a session store.

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Essentially, SOA is an architectural style where software components (services) communicate over a network via standardized protocols. It has been defined to emphasize reusability, loose coupling, and interoperability.

Service. *A self-contained, modular software component that performs a specific function and communicates with other services over a network.*

Each service provides a repeatable business activity (e.g., “Recommendation Service”). It may be composed of other auxiliary services.

Services communicate between them via an Enterprise Service Bus (ESB).

In SOA, a service may either be *stateful* or *stateless*:

A *stateless* service does **not** retain any data between requests. For instance, a **machine learning inference service** that classifies text but does not remember previous inputs.

As it does not retain any data, it requires clients to send all necessary data in each request. However, multiple instances can handle requests independently.

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Besides the concept of service, SOA includes two main components enabling the communication between services and between users and services.

The *service registry* is a centralized registry maintains metadata about available services, enabling clients to dynamically locate services instead of hardcoding their locations.



Client-Service Communication

A service registers itself with a **service registry** upon deployment, providing metadata such as host, port, and health check endpoint. Some registries require periodic **heartbeats** to maintain an active registration.

A client queries the **registry** to retrieve a list of active service instances. The client then sends a request, either directly to a selected service instance or through an **Enterprise Service Bus (ESB)**, which manages routing and mediation.

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Besides the concept of service, SOA includes two main components enabling the communication between services and between users and services.

The *enterprise service bus* is a middleware that acts as a central communication hub, routing and transforming messages between services.

Service-Service Communication



Services send requests to the **Enterprise Service Bus**, which routes them to the appropriate service. The ESB handles **data transformation** (e.g., converting XML to JSON), **security enforcement, logging, and monitoring**.

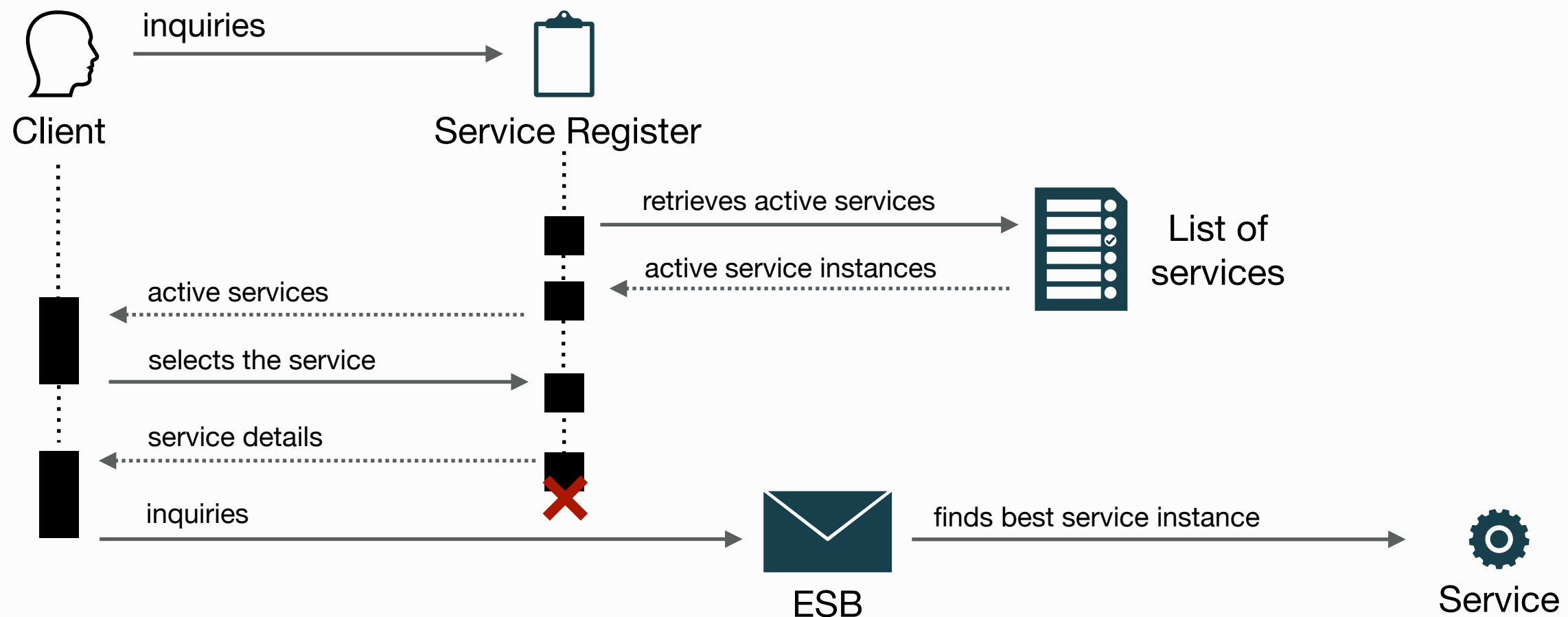
While the ESB simplifies integration by **decoupling services**, making them easier to replace or upgrade, it can also introduce **performance bottlenecks** and act as a **single point of failure** if not properly managed. Additionally, the ESB **bridges protocol differences**, enabling interoperability between services using different technologies.

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

So, to summarize:



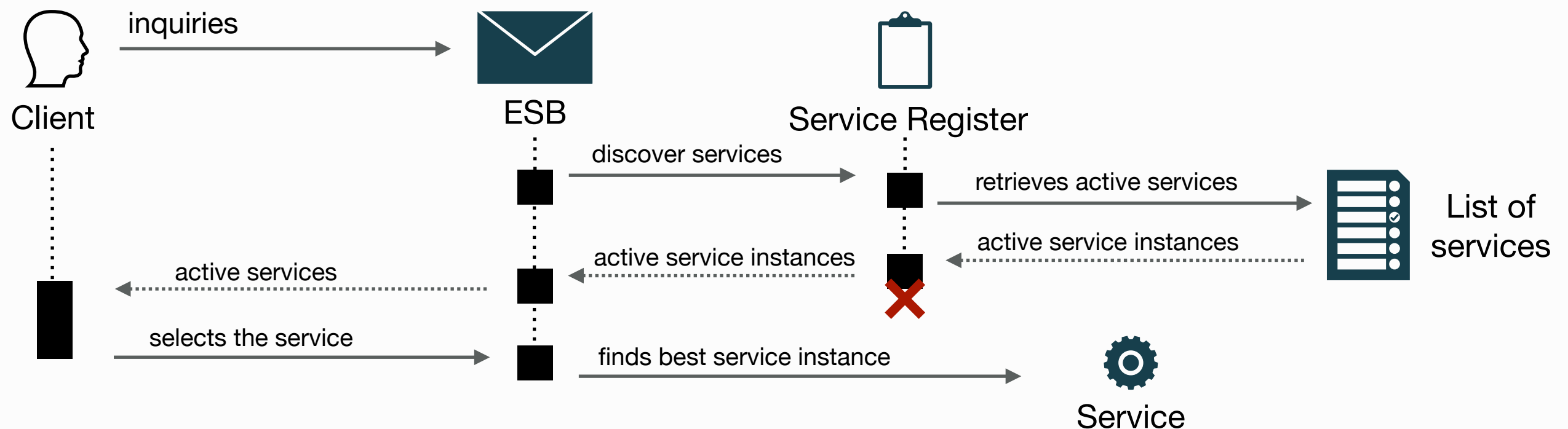
Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

So, to summarize:



In some SOA architectures, the **ESB itself** is responsible for discovering services rather than the client. If so, the **ESB would query the Service Register** instead of the client doing it manually.

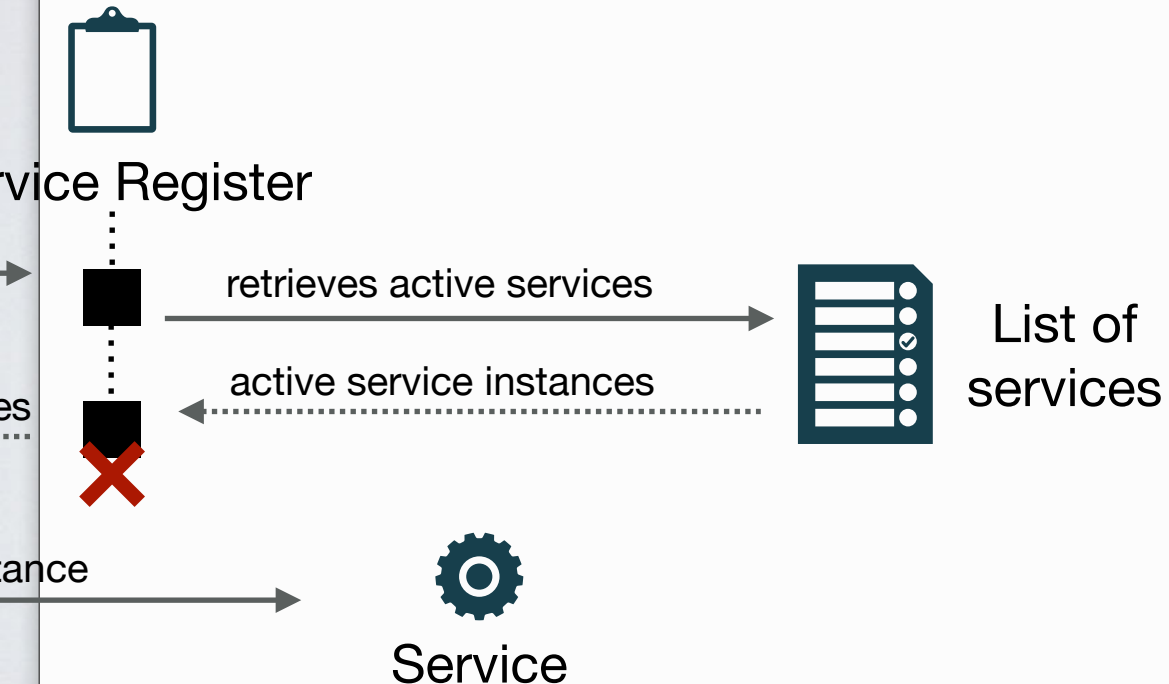
Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

So, to summarize:



In this formulation, we can easily see some key advantages:

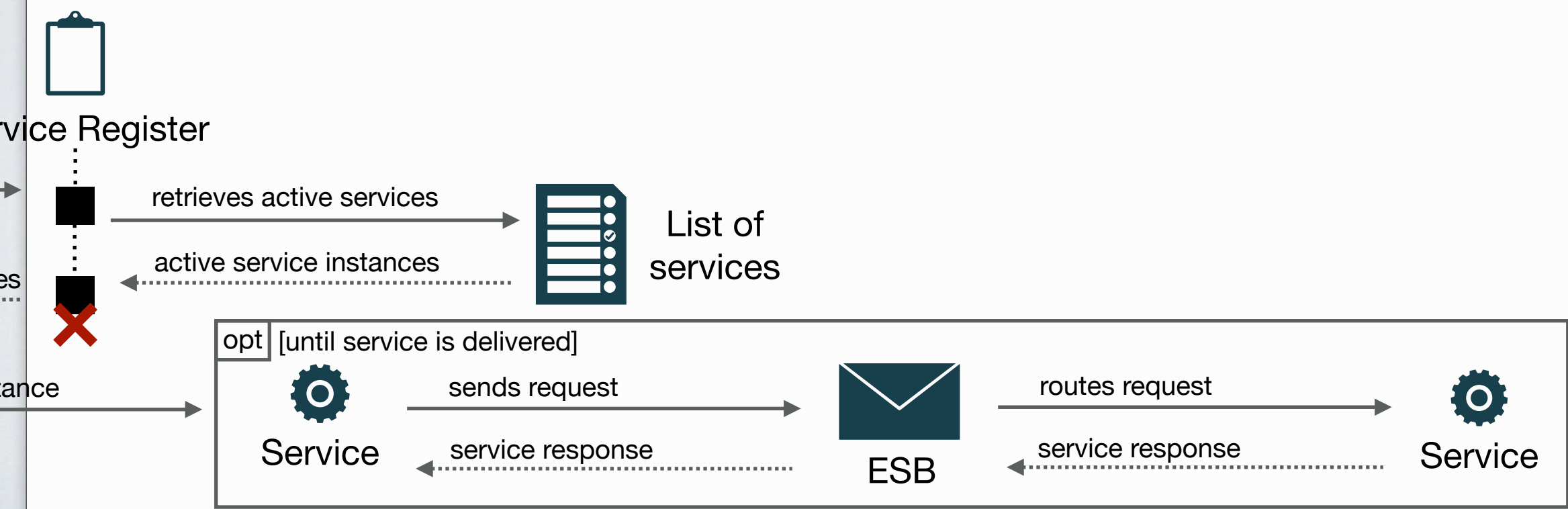
1. Services **can be reused** across multiple applications, e.g., the same model service can serve more use cases;
2. Individual services can be **scaled independently** based on demand, e.g., a NLP processing service can be instantiated multiple times to handle more queries.

discovering services rather than the client. If so, client doing it manually.

Thinking like an Architect: Common Architectural Challenges and System Structures

- 3 **Service-oriented architectures and microservices.** A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

So, to summarize:



Thinking like an Architect: Common Architectural Challenges and System Structures

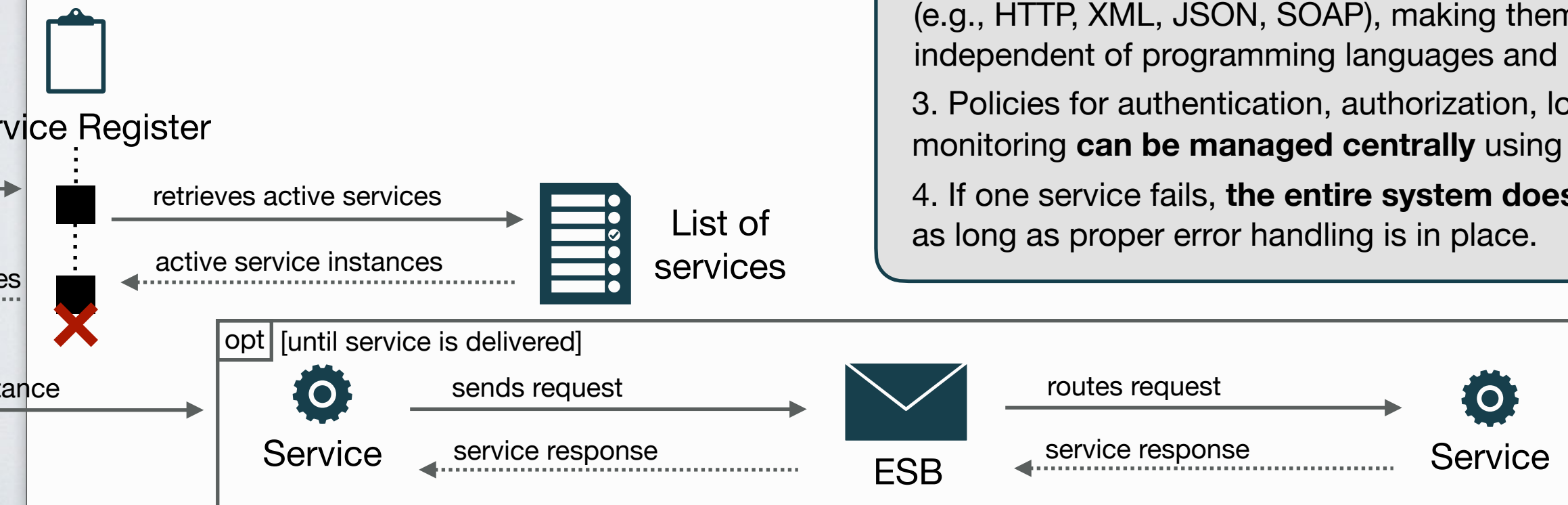
3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

So, to summarize:

Additional advantages:

1. Services **are decoupled**, e.g., a **ML model** can be swapped out with a new version without affecting others.
2. Services communicate using **standardized protocols** (e.g., HTTP, XML, JSON, SOAP), making them independent of programming languages and platforms.
3. Policies for authentication, authorization, logging, and monitoring **can be managed centrally** using the ESB.
4. If one service fails, **the entire system does not crash**, as long as proper error handling is in place.



Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Microservices extend SOA principles by making services **smaller, independent, and deployable separately**. Specifically, each microservice:

1

The scope of a microservice is finer-grained. It is not a *modular software component* anymore, but something smaller than implements a specific function.

Microservice. A *small, independent, and self-contained software component* that performs a specific function, communicates via lightweight APIs, and can be independently deployed and scaled within a distributed system.

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Microservices extend SOA principles by making services **smaller, independent, and deployable separately**. Specifically, each microservice:

1

The scope of a microservice is finer-grained. It is not a *modular software component* anymore, but something smaller than implements a specific function.

To better understand the difference, consider a recommendation service that provides (1) User behavior analysis; (2) Product recommendations; and (3) Real-time personalization.

In many SOA implementations, these AI models would often be grouped within a **single business service** (e.g., a "Recommendation Service") because:

- The ESB routes messages between services but introduces overhead;
- Service boundaries in SOA tend to be larger (coarse-grained), meaning multiple AI models may be wrapped within a single service.
- Collaboration often happens within a service before exposing a final API.

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Microservices extend SOA principles by making services **smaller, independent, and deployable separately**. Specifically, each microservice:

1

The scope of a microservice is finer-grained. It is not a *modular software component* anymore, but something smaller than implements a specific function.

To better understand the difference, consider a recommendation service that provides (1) User behavior analysis; (2) Product recommendations; and (3) Real-time personalization.

In many SOA implementations, these AI models would often be grouped within a **single business service** (e.g., a "Recommendation Service").

On the contrary, a microservice-approach would likely have a "*User Behavior Analysis Microservice*" that analyzes browsing habits, a "*Product Recommendation Microservice*" that applies collaborative filtering, and a "*Personalization Microservice*" that fine-tunes recommendations based on real-time interactions.

How can this approach be more scalable then?

Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

Microservices extend SOA principles by making services **smaller, independent, and deployable separately**. Specifically, each microservice:

1

The scope of a microservice is finer-grained. It is not a *modular software component* anymore, but something smaller than implements a specific function.

2

Each microservice *has its own database* (i.e., this architecture implements the “Polyglot Persistence” concept) instead of a shared database.

3

Microservices can bypass the ESB, opting for direct service-to-service communication using HTTP. This reduces the risk of ESB becoming a bottleneck.

These two properties have key implications. First, collaboration happens **via direct API calls** or **event-driven messaging**, NOT through a centralized ESB. This makes the architecture faster and more flexible.

Second, as each microservice has its own database, this means that it **can be fully isolated**, i.e., each microservice can be deployed independently from the others. This implies even more independent failures than traditional SOA architectures.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

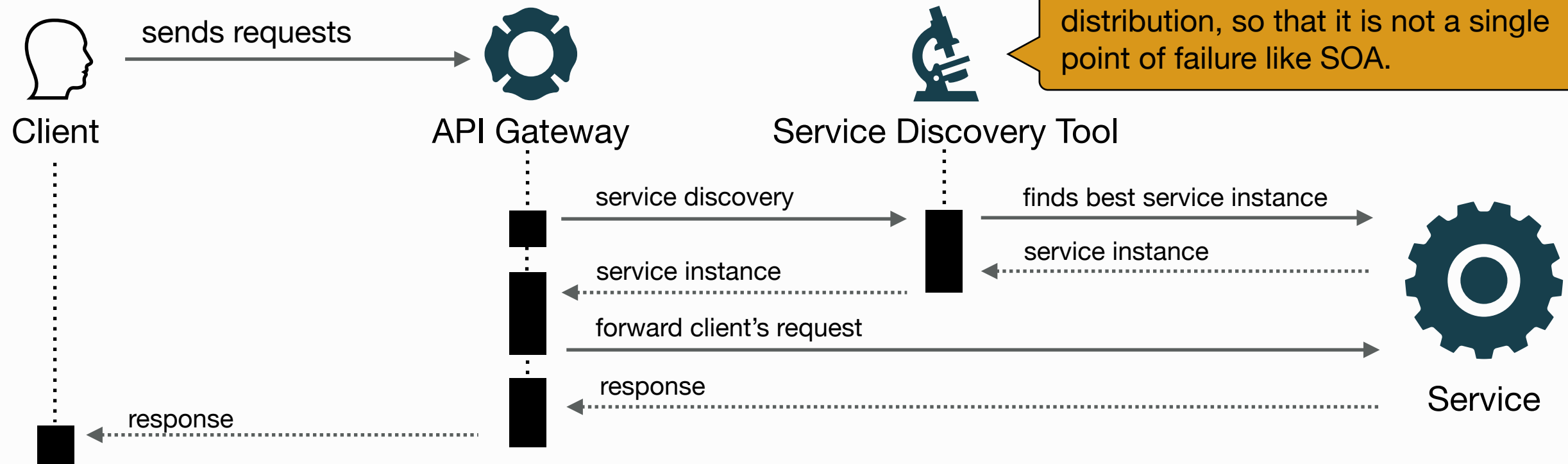
3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

So, to summarize:

Services get **registered** in the discovery tool, i.e., in tools like Eureka, Consul, or Kubernetes.

Service discovery is **decentralized**: this tool supports replication and distribution, so that it is not a single point of failure like SOA.



Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

To make the difference between SOA and microservices even clearer, let's consider the code required to query a ML service.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

SOA Architecture

```
from zeep import Client

# Step 1: The client prepares the request by setting the ESB's WSDL URL
wsdl_url = "http://localhost:8000/?wsdl"

# Step 2: The client initializes a SOAP client that communicates with the ESB
print("Client: Connecting to ESB at", wsdl_url)
client = Client(wsdl_url)

# Step 3: The client sends a request to the ESB
print("Client: Sending request to ESB for prediction...")
response = client.service.predict(3.5, 1.2) # Actual SOAP request

# Step 4: The ESB processes the request and finds the best ML service,
# then forwards the request and returns a response to the client.
print("Client: Received response from ESB ->", response)
```

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

SOA Architecture

```
from zeep import Client
```

```
# Step 1: The client prepares the request by setting
```

```
wsdl_url = "http://localhost:8000/?wsdl"
```

Clients must explicitly connect and interact with the ESB

```
# Step 2: The client initializes a SOAP client that communicates with the ESB
```

```
print("Client: Connecting to ESB at", wsdl_url)
```

```
client = Client(wsdl_url)
```

```
# Step 3: The client sends a request to the ESB
```

If this object is busy, this request will take a while!

```
print("Client: Sending request to ESB for prediction")
```

```
response = client.service.predict(3.5, 1.2)
```

Clients need to know the interface of the service on the ESB

```
# Step 4: The ESB processes the request and finds the best ML service,
```

```
# then forwards the request and returns a response to the client.
```

```
print("Client: Received response from ESB ->", response)
```

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

Microservices

```
import requests
```

```
# Step 1: The client prepares the request
```

```
api_gateway_url = "http://localhost:8080/predict"
```

```
params = {"feature1": 3.5, "feature2": 1.2}
```

Clients just need to know where the API of the model is.

```
# Step 2: The client sends the request to the API Gateway
```

```
response = requests.get(api_gateway_url, params=params)
```

Then, they send a parameterized request through HTTP - there are no intermediaries involved.

The distributed mechanism, hidden to the client, allows to load balance the requests, avoiding bottlenecks.

```
# Step 3: The API Gateway processes the request, finds the best ML service,  
# then forwards the request to the selected ML microservice.
```

```
# Step 4: The client receives the response from the API Gateway
```

```
print("Client: Received response from API Gateway ->", response.json())
```

They receive a response in JSON (typically)

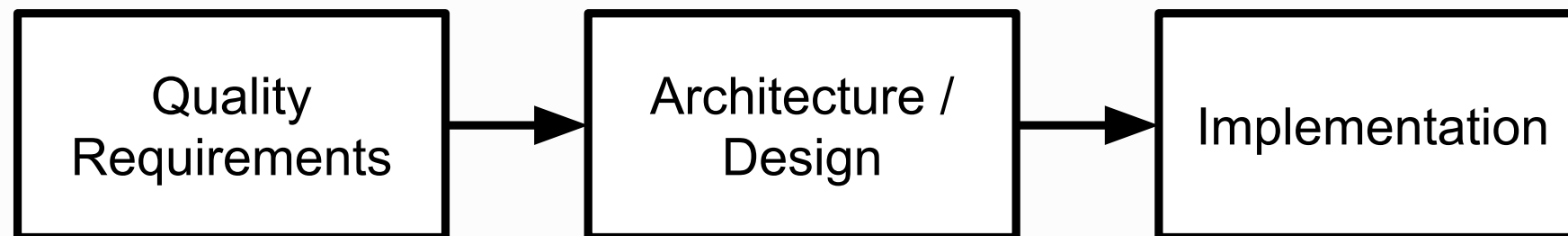
Thinking like an Architect: Common Architectural Challenges and System Structures

3

Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

To make the difference between SOA and microservices even clearer, let's consider the code required to query a ML service.

The example makes clear why and how architectural design strongly impacts the way the system will be implemented. Remember where we started:



The choice of architecture depends on the system's needs: scalability, modularity, performance, and ease of deployment. Aligning architectural design with quality requirements ensures an efficient, maintainable, and performant ML system.

That said, let's discuss how can different quality requirements inform the decision about the best architectural style to use within specific contexts.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures

Software engineers often refer to quality attributes, quality requirements, or non-functional requirements (design goals) to describe desirable system properties, e.g., latency, safety, and usability, that serve service-level objectives.

Data scientists sometimes speak of model properties when referring to accuracy, inference latency, fairness, and other qualities of a learned model, but that term is also used for capabilities of the learning algorithm.

Performance-related quality attributes

Inference Latency. The time required to generate a single prediction.

Low latency models may require local processing (aligning with Client-Server architectures).

Cloud-based architectures (SOA, Microservices) add network latency.

Inference Throughput. The number of predictions per second.

Batch processing tasks (e.g., transcription services) require Multi-Tier with queues.

Large-scale real-time inference (e.g., music recommendations) is best suited for Microservices.

Scalability. How well the system handles increasing workloads.

SOA (ESB) scales at enterprise level but introduces bottlenecks.

Microservices scale independently for real-time systems (e.g., recommendation engines).

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures

Software engineers often refer to quality attributes, quality requirements, or non-functional requirements (design goals) to describe desirable system properties, e.g., latency, safety, and usability, that serve service-level objectives.

Data scientists sometimes speak of model properties when referring to accuracy, inference latency, fairness, and other qualities of a learned model, but that term is also used for capabilities of the learning algorithm.

Model-specific quality attributes

Prediction accuracy. The extent to which the model correctly predict outcomes.

Model Size & Memory Footprint. The storage space required to save a machine learning model and the RAM needed to load and execute it during inference or training.

Large models require cloud-based inference (SOA, Microservices).

Small models can be deployed on-device (Client-Server) for privacy-focused applications (e.g., smart keyboards).

Energy Efficiency. The ability of a model or system to minimize power consumption while maintaining performance, especially in resource-constrained environments like edge devices and mobile applications.

Critical for edge AI and battery-powered devices. Smartphones running federated learning must minimize power consumption (Client-Server).

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures

Software engineers often refer to quality attributes, quality requirements, or non-functional requirements (design goals) to describe desirable system properties, e.g., latency, safety, and usability, that serve service-level objectives.

Data scientists sometimes speak of model properties when referring to accuracy, inference latency, fairness, and other qualities of a learned model, but that term is also used for capabilities of the learning algorithm.

Maintainability and Evolution

Training Latency. The time required to retrain a model.

Frequent retraining requires automated pipelines (Multi-Tier, Microservices).

Model Versioning & Deployment Latency. How quickly a new model version can be deployed.

Microservices allow CI/CD, enabling fast updates without disrupting the system.

Observability & Monitoring. The ability to track, analyze, and maintain the performance, reliability, and behavior of ML models and systems in production through metrics, logs, and alerts.

Most architectures enabled the model to be tracked over time - MLOps would cope with this need, providing a framework to continuously monitor the system performance.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures

Software engineers often refer to quality attributes, quality requirements, or non-functional requirements (design goals) to describe desirable system properties, e.g., latency, safety, and usability, that serve service-level objectives.

Data scientists sometimes speak of model properties when referring to accuracy, inference latency, fairness, and other qualities of a learned model, but that term is also used for capabilities of the learning algorithm.

Security, Privacy, and Fairness

Privacy & Data Protection. The extent to which the model safeguards user data by ensuring secure storage, access control, anonymization, and compliance with regulations.

Client-Server is best for on-device, privacy-preserving ML (e.g., smart keyboards).

SOA and Microservices require secure data governance.

Explainability & Interpretability. The ability to understand, trust, and explain how a machine learning model makes decisions, ensuring transparency and accountability.

Simple models (decision trees) are **inherently interpretable**.

SOA is better for integrating human-in-the-loop review systems.

Fairness & Bias Mitigation. The ability of the model to make unbiased, equitable decisions by detecting, reducing, and preventing discrimination against specific groups.

Requires auditing tools in ML pipelines (Multi-Tier, Microservices).

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures

Software engineers often refer to quality attributes, quality requirements, or non-functional requirements (design goals) to describe desirable system properties, e.g., latency, safety, and usability, that serve service-level objectives.

Data scientists sometimes speak of model properties when referring to accuracy, inference latency, fairness, and other qualities of a learned model, but that term is also used for capabilities of the learning algorithm.

Cost & Trade-offs

Cost per Prediction. The economic cost of each prediction made by the deployed model.

High inference costs (e.g., GPT-3) require optimized cloud inference (Microservices, Multi-Tier).

Low-cost, high-volume models may use two-stage processing (SOA, Microservices).

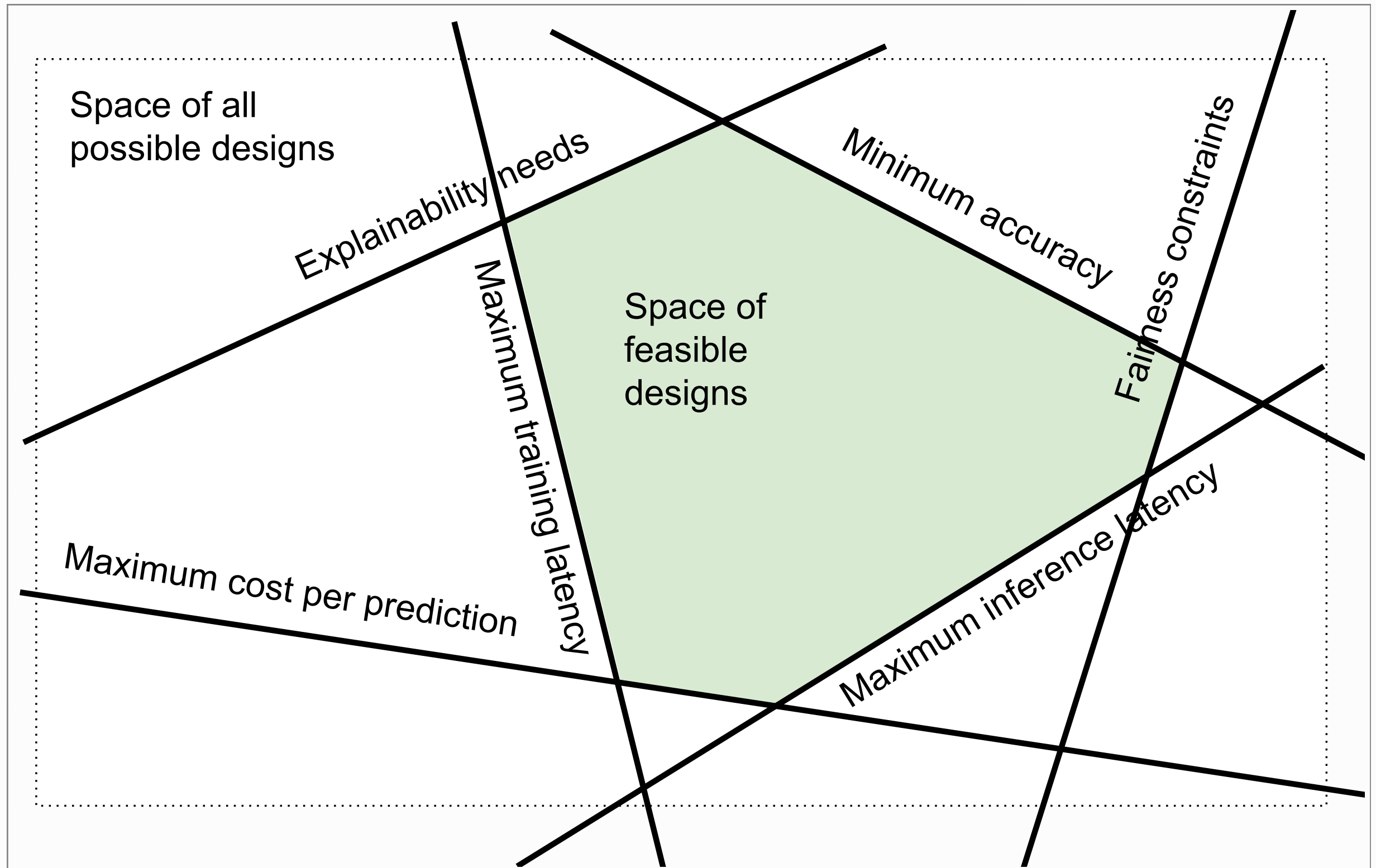
Infrastructure Cost. The cost due to hardware, storage, and networking.

SOA requires expensive centralized infrastructure (ESB, service registry).

Microservices allow pay-as-you-go cloud scaling.

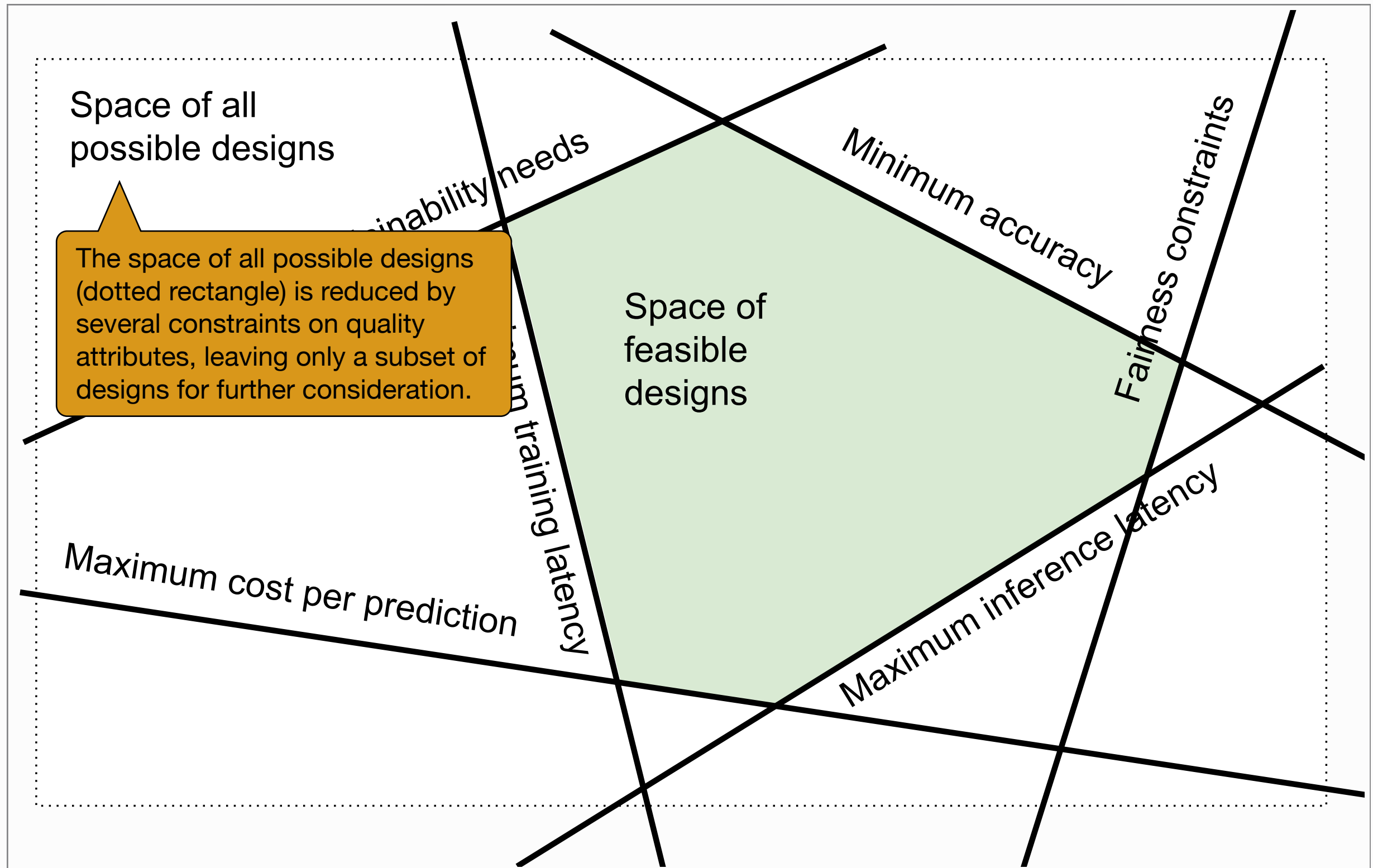
Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures



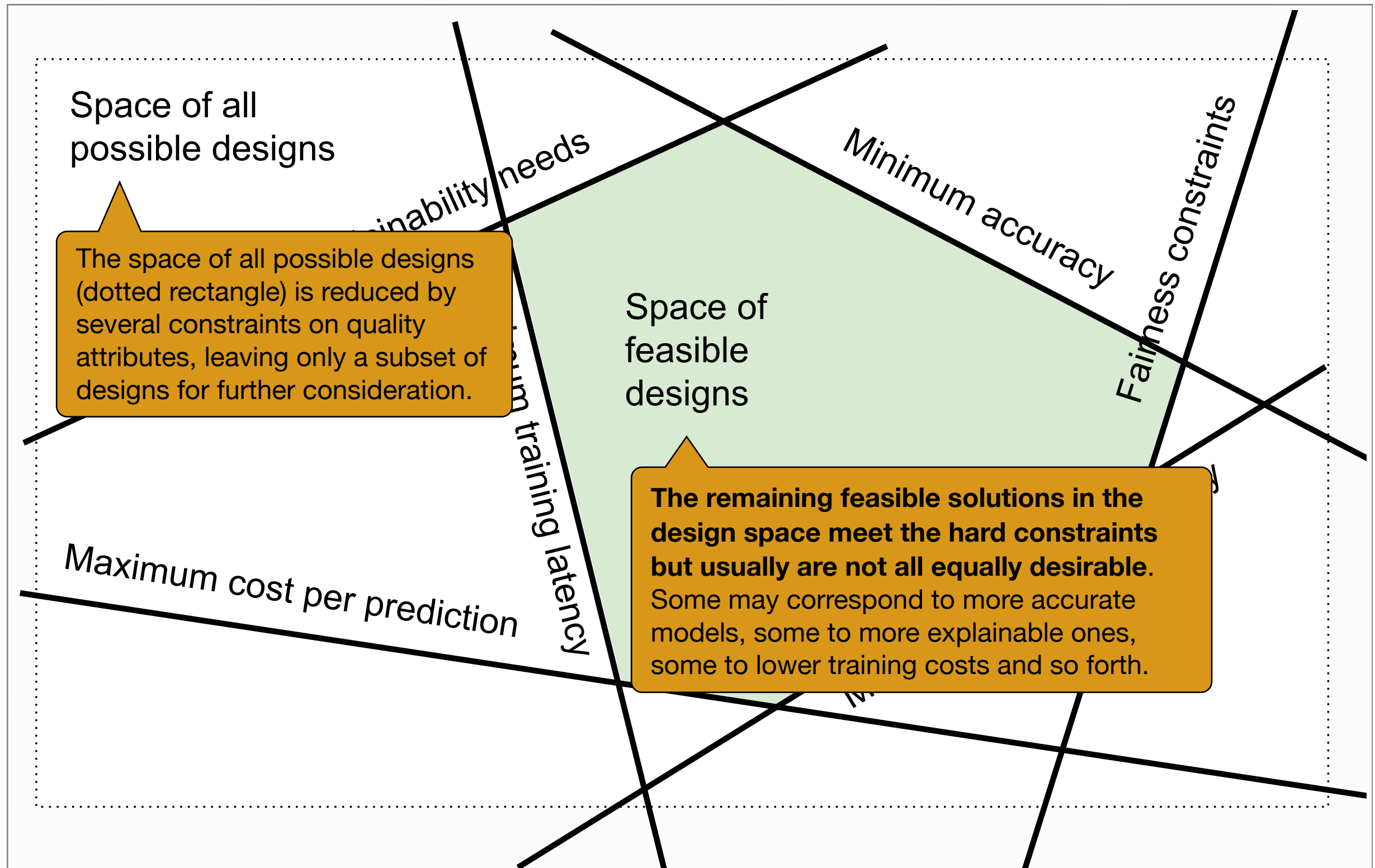
Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

Production systems that use AI will at some point deploy one or more AI models and use them to make predictions. The step of using an AI model to make a prediction for some given input data is typically called *model inference*.

While seemingly simple, there are lots of design decisions and trade-offs when considering how to deploy a model, depending on system requirements, model qualities, and interactions with other components.

A common form is to provide a *model inference service as a microservice*, called through remote procedure calls by other components of the system, but models may also be embedded as *libraries* on client devices or even deployed as *precomputed tables*.

Let's consider the scenario of an Augmented Reality Translation.

Consider a big tech company wanting to give smart glasses another chance. Those glasses can record audio and video and project images with an optical head-mounted display.

Smart glasses aim for *augmented reality*, where information is overlaid over the real world.

To support the upcoming release of the new smart glasses, the company wants to develop an AR-translation app that can translate text in the real world, for example, translating signs while walking through a city in a foreign country as a tourist.



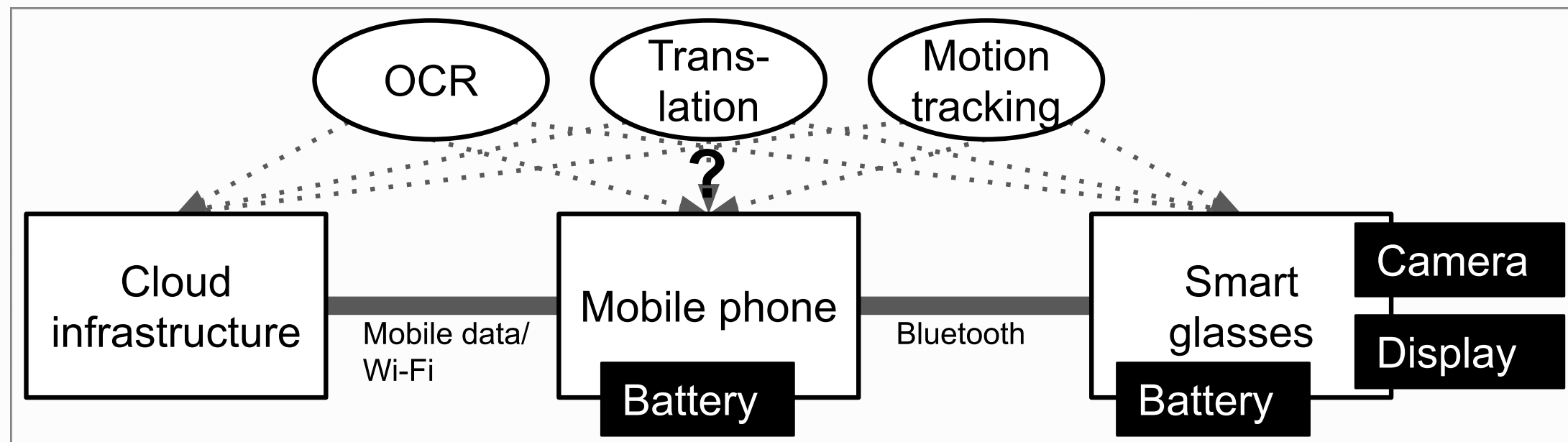
Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

Let's consider the scenario of an Augmented Reality Translation.

There are many existing building blocks for such an application. *Text translation* is standard these days, as is optical character recognition (OCR). There are also several products, including Google Translate for smartphones that can already automatically recognize and translate text in live video, overlaying the translation over the original text in a live camera feed. Also, image stabilization modules are well researched.

To build an augmented reality translation app for smart glasses then, we need to decide **how to customize, integrate, and deploy the various components of the system**. The glasses themselves **may not have enough computation power and battery to run all models**, but they can communicate with a smartphone over Bluetooth, which may talk to a cloud-backed server over Wi-Fi or a cellular connection.



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

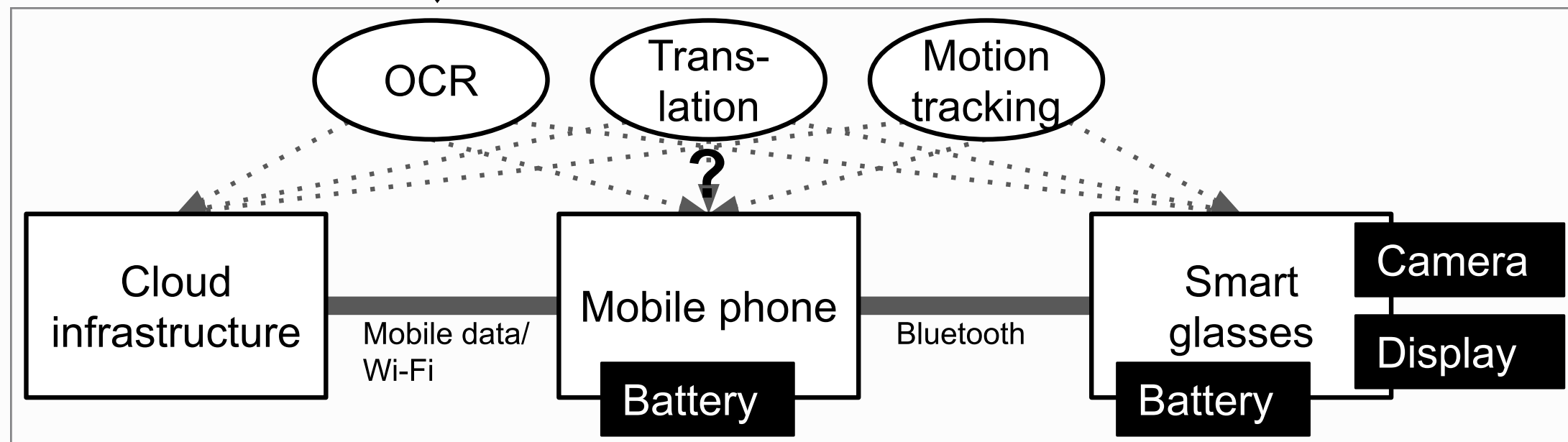
Let's consider the scenario of an Augmented Reality Translation.

There are many existing building blocks for such an application. *Text translation* is standard these days, as is optical character recognition (OCR). There are also several products, including Google Translate for smartphones that can already automatically recognize and translate text in live video, overlaying the translation over the original text in a live camera feed. Also, image stabilization modules are well researched.

To build an augmented reality translation app for smart glasses then, we need to decide **how to customize, integrate, and deploy the various components of the system**. The glasses themselves **may not have enough computation power and battery to run all models**, but they can communicate with a cloud server over Wi-Fi or a cellular network.

An architectural model of the augmented reality translation scenario, where three models could be deployed on different hardware.

An architectural model of the augmented reality translation scenario, where three models could be deployed on different hardware.



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

At their bare essentials, most AI models have a really simple interface for prediction, which can be encapsulated as a simple side-effect-free function. They take as input a *feature vector of values* and produce as output a *prediction* using the previously learned model.

The feature vector is a sequence of numbers representing the input, say, the colors of pixels of an image or numbers representing characters or words.

The inference computations of the model usually follow a relatively simple structure composed of matrix multiplications, additions, and if statements, often in long sequences, with the constants and cutoffs (the model parameters) learned during model training.

In our augmented-reality-translation scenario, the OCR model might take an image represented as a vector of numbers and return a probability score and bounding box for each detected character.

Models are typically stored in a serialized format in files and are loaded into memory when needed for inference tasks. These kinds of inference functions can now be used by other parts of the system to make predictions.

```
from sklearn.linear_model import LogisticRegression
model = ... # learn model or load serialized model ...
def infer(feature1, feature2):
    return model.predict(np.array([[feature1, feature2]]))
```

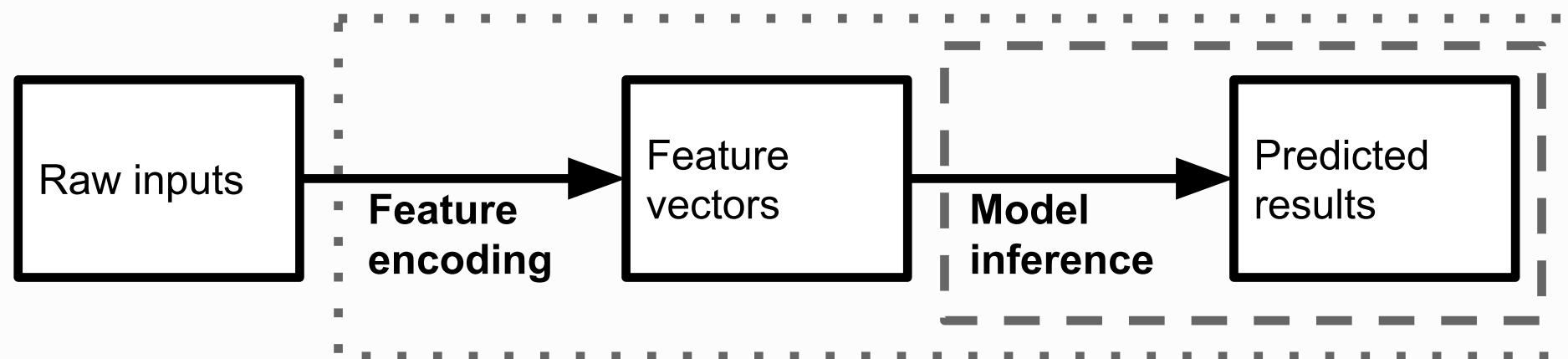
For instance, this is an inference function for a simple classification model taking two numeric feature inputs in scikit learn.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

In almost all settings, there is a feature encoding step that takes the original input, such as a JPEG image, a GPS coordinate, a sentence, or a row from a database, and converts it into the feature vector in the format that the model expects.

Feature engineering can be simple, such as just normalizing a value representing a person's age to a range between -1 and 1. It can also involve more sophisticated computations, doing nontrivial transformations, nontrivial data aggregation, or even applying other AI models.



Feature encoding is an important step in the inference process that may happen within (dotted box) or outside (dashed box) the scope of the inference function.

When thinking of model inference as a function within a system, feature encoding can (1) happen with the model-inference function or (2) be the client's responsibility. That is, the client provides either raw inputs such as image files or feature vectors to the inference service.

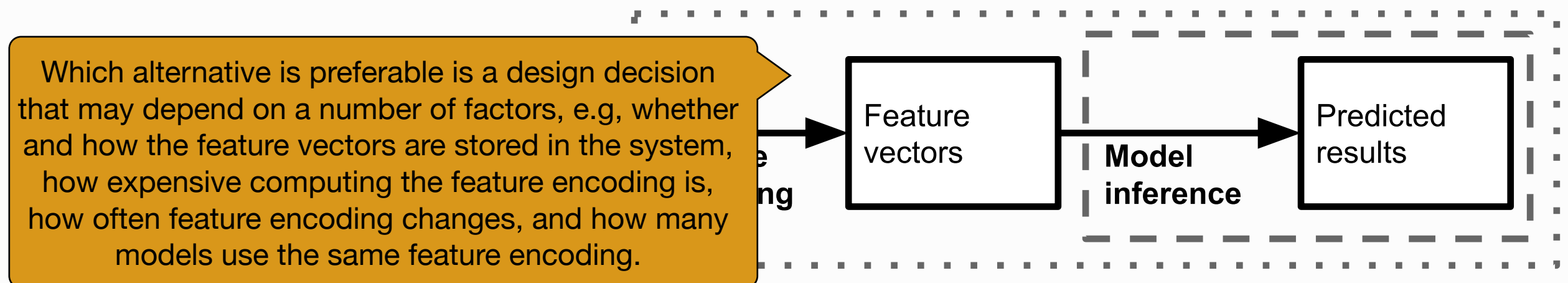
Feature encoding and model inference could even be *two separate services* that are called in sequence by the client.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

In almost all settings, there is a feature encoding step that takes the original input, such as a JPEG image, a GPS coordinate, a sentence, or a row from a database, and converts it into the feature vector in the format that the model expects.

Feature engineering can be simple, such as just normalizing a value representing a person's age to a range between -1 and 1. It can also involve more sophisticated computations, doing nontrivial transformations, nontrivial data aggregation, or even applying other AI models.



Feature encoding is an important step in the inference process that may happen within (dotted box) or outside (dashed box) the scope of the inference function.

When thinking of model inference as a function within a system, feature encoding can (1) happen with the model-inference function or (2) be the client's responsibility. That is, the client provides either raw inputs such as image files or feature vectors to the inference service.

Feature encoding and model inference could even be *two separate services* that are called in sequence by the client.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

A similar discussion also applies to post-processing of prediction results, as common to normalize outputs when using large language models for classification tasks. Such post-processing can happen within or outside of the inference service.

For more sophisticated processing of outputs with some business logic, it is more common though to separate that processing into a separate module.

In any case, the same feature encoding must be used during training and inference.

Inconsistencies are dangerous and can lead to wrong predictions as the runtime data does not match the training data. This can span from minor inconsistencies, such as inconsistent scaling of a single feature, to completely scrambling the inputs, like changing the order in which colors are encoded changes.

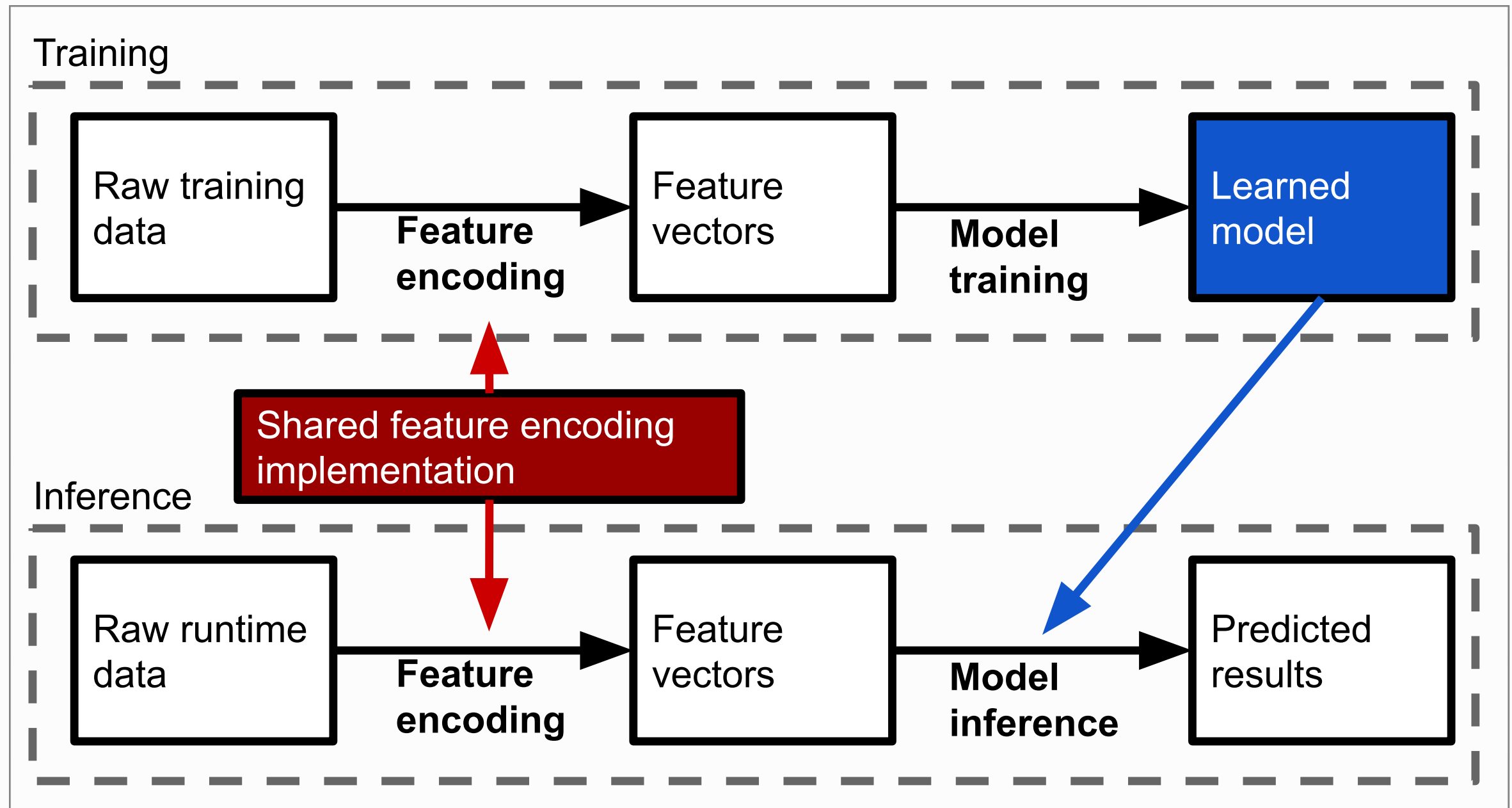
To prevent inconsistent encoding between training and inference, known as *training-serving skew*, a common practice is to write feature encoding code as a function used both in training and at inference time.

This could be as simple as a Python function imported from both the training and inference code. That code must be versioned together with the model.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

We can visualize the discussion so far through this chart



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

In large organizations, it may also be desirable to collect and reuse code for feature extraction and feature encoding so that it can be reused across different projects. For example, multiple models may need to recognize local languages from GPS coordinates. Rather than having multiple teams reimplement the same/similar feature extraction/encoding implementations, it can be beneficial to collect them in a central place and make them easily discoverable.

This is particularly true for more challenging encodings that may involve other AI models, such as using a saliency model to decide how to crop an image, and encodings using nontrivial database queries. Documenting the different pieces of feature engineering code may further help to discover and reuse them across multiple models and projects.

In addition, if features are reused across multiple projects or need to be extracted from the same data repeatedly over time, it may be worth precomputing or caching the feature vectors. For example, multiple models may need similar features computed for all our registered users.

Many companies adopt **feature stores** to solve both problems. Feature stores catalog reusable feature engineering code and can help to (pre-)compute and cache feature vectors, especially when operating with large amounts of data and expensive feature encoding operations.

There are many competing open-source and commercial projects, such as Feact, Tecton, AWS SageMaker Feature Store, and others.

In other terms, a system thinking - or even an organizational thinking - may inform the way we want to deploy services required by an AI-enabled system!

Thinking like an Architect: Model Serving Infrastructure

Feature encoding code and model inference are almost always free of side effects (also called *stateless* or *pure*): they do not store any information between multiple invocations, so all function invocations are independent.

The lack of side effects makes it easy to parallelize feature encoding and inference code, answering separate requests from different processes without any required synchronization—the problem is **embarrassingly parallel**.

Common forms of serving a model - at least four designs:

1

Model inference as a library. The model and inference function can simply be embedded as a local function or library in the target system. The model is loaded into memory, and inference is performed with a local function call.

In this case, the model may be shipped as part of the system binaries or loaded over the network. In addition, various libraries facilitate embedding serialized models, such as TensorFlow Lite for mobile apps and TensorFlow.js for client-side browser deployments.

2

Model inference as a service. The most common form of deploying an AI model is to provide a dedicated server that responds to networked client requests over a REST API.

Depending on the model, we might benefit from specialized hardware for the service. To scale the system, multiple instances of such model inference service can be distributed across multiple machines, locally or in some cloud infrastructure, often using some container virtualization.

Thinking like an Architect: Model Serving Infrastructure

Feature encoding code and model inference are almost always free of side effects (also called *stateless* or *pure*): they do not store any information between multiple invocations, so all function invocations are independent.

The lack of side effects makes it easy to parallelize feature encoding and inference code, answering separate requests from different processes without any required synchronization—the problem is **embarrassingly parallel**.

Common forms of serving a model - at least four designs:

3

Batch processing. If the model is smaller than the data, it is more efficient to perform model inference where the data is stored rather than sending data to an inference service. Batch processing can move a model to the data and perform inference in bulk.

4

Cached predictions. When predictions are expensive but not many distinct inputs need to be predicted, it can be worth caching or even precomputing all predictions. This way, inference can be reduced to looking up previous predictions of the model in some database—such as translations for common words.

Of course, hybrid approaches for all of these exist. For instance, consider the case where a model is deployed both as a *local library* and a *service*, allowing for low-latency local inference when resources permit but falling back to a centralized service when greater computational power is required.

Thinking like an Architect: Model Serving Infrastructure

While the technical deployment of model inference as a library or service is often straightforward, deciding whether to deploy as a library or service and where to deploy can require nontrivial trade-offs in many systems.

First, we distinguish between **locations** where the product's code and model inference will be executed. For simplicity, we consider (1) **servers** under the control of the application provider, whether in-house servers or cloud infrastructure, and (2) a **user-facing device** through which the user interacts with the system, such as a desktop computer, a mobile phone, a public terminal, a smart-home appliance, or our smart glasses.

1

Server-side model deployment. This happens when the model inference is deployed as a service on a server. Server-side deployment makes it easy to provide *dedicated resources for model inference* and scale it independently.

This can be particularly useful if models are large or have resource demands on memory, energy, and GPU that exceed those of the user-facing devices. With large foundation models, server-side deployment is typically the only option.

Server-side deployment places *all computational costs for model inference on the server infrastructure*, rather than distributing them to the user-facing devices.

For this approach to work, the user-facing devices need a *network connection* to the server. Hence, model inference will not work offline, and network latency and bandwidth can be a concern. If inputs for model inference are large, e.g., images, this design can cause significant network traffic, both for clients and server operators.

Thinking like an Architect: Model Serving Infrastructure

While the technical deployment of model inference as a library or service is often straightforward, deciding whether to deploy as a library or service and where to deploy can require nontrivial trade-offs in many systems.

First, we distinguish between **locations** where the product's code and model inference will be executed. For simplicity, we consider (1) **servers** under the control of the application provider, whether in-house servers or cloud infrastructure, and (2) a **user-facing device** through which the user interacts with the system, such as a desktop computer, a mobile phone, a public terminal, a smart-home appliance, or our smart glasses.

2

Client-side model deployment. This happens when the model inference is deployed on the client-facing device, typically bundled with the application as a binary resource.

Since no network connection is needed, model inference can be performed *offline*, and there are no concerns about network latency or bandwidth. Offline, on-device inference is also commonly seen as *protecting privacy*, since user data is not (necessarily) leaving the user-facing device.

When models are deployed client-side, the target devices need to have sufficient *computational resources* for model inference, which might be a significant concern for a company. Also, if the hardware of client devices is heterogeneous, e.g., applications to be deployed on many different mobile phones, inference latency and user experience may vary substantially between users, leading to inconsistent user experiences.

Thinking like an Architect: Model Serving Infrastructure

While the technical deployment of model inference as a library or service is often straightforward, deciding whether to deploy as a library or service and where to deploy can require nontrivial trade-offs in many systems.

Besides location, decisions about where to deploy a model can significantly influence **how the model can be updated** and how easy it is to **experiment** with different model variants.

1 Server-side updates. For server-side model deployments, it is easy to deploy a new version of a model without having to modify the clients. If the input and output types of the different model versions are compatible, developers can simply replace the model behind an API endpoint.

This flexibility to update the model independently from the application is also routinely used to experiment with different model variants in production.

2 Client-side updates. For client-side model deployments, updates and experimentation are more challenging and may require more planning. The key problems are (1) that updates may require transferring substantial amounts of data to the user-facing devices, especially with large models, and (2) that the devices may not all be connected with equally reliable/ fast network connections and may not always be online.

In this case, we may decide to never update the model, to let users update the entire applications, or to separate model updates from application updates (e.g., our translation app may download and update translation models for individual languages without updating the app or other models).

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

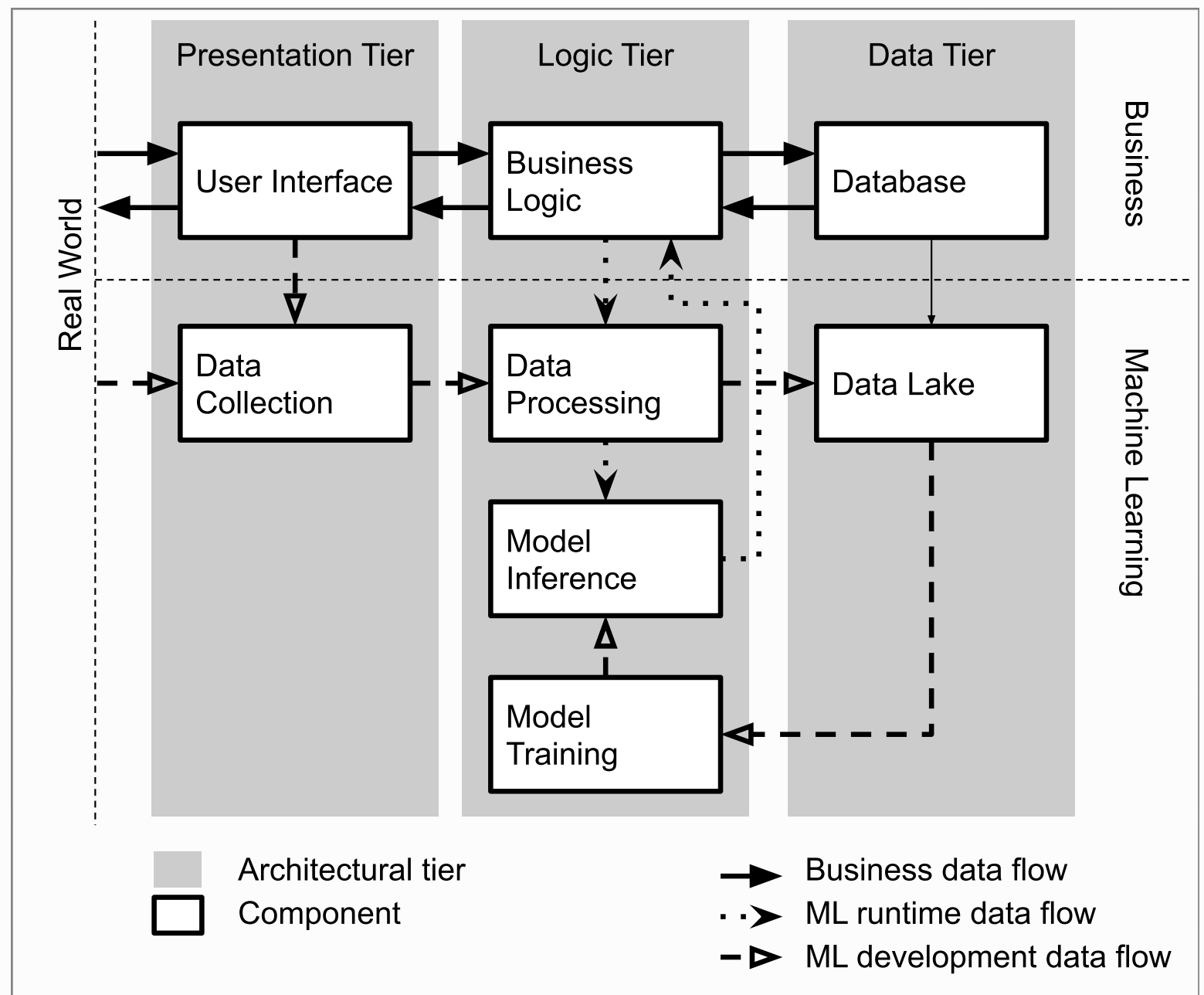
Thinking like an Architect: Model Inference in a System

Model inference is an important component of an ML-enabled system, but it needs to be integrated with other components of a system, including non-ML components that request and use the predictions. It is worth deliberately structuring the system.

Designers typically strive to separate concerns. When it comes to machine learning, it is common to clearly separate models from business logic within the system.

Such separation, if done well, makes it easier to encapsulate change and evolve key parts of the system independently.

Similarly, the pipeline infrastructure for training a model and the infrastructure for collecting telemetry and training data can, and usually should, be separated from the non-ML parts of the system to isolate ML-related code and infrastructure and evolve it independently.



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Documenting Model Inference Interfaces

Whether provided as a library or web service, a model-inference service has an interface that clients can call to get predictions. In many systems, this interface is at the boundary between different teams—between the team that trains and operates the model and the team that builds the software using the model.

In all cases, documentation at this interface is important, though often neglected. Using a model without understanding its quality or limits can lead to unfortunate quality problems.

For traditional software components, interface documentation is common. Traditionally, interface descriptions include the type signature and a description of the functionality of each function in the interface. As for AI models, it is fairly standard to provide a technical description of how to invoke a model inference function or service.

```
{  
  "mid": string,  
  "languageCode": string,  
  "name": string,  
  "score": number,  
  "boundingPoly": {  
    object (BoundingPoly)  
  }  
}
```

For instance, this is an excerpt of the Google's public object detection API. The API describes the URL for the endpoint, authentication, and the format in which the request need to be sent (JSON object in which images are base64-encoded strings) and that the API will return identified objects as LocalizedObjectAnnotation object shown in this figure, describing the object, confidence, and bounding box.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Documenting Model Inference Interfaces

An increasing alternative is called **model card**.

It is brief (one- to two-page) description of models, including intended use, training and evaluation data, considered demographic factors, evaluation results broken down by demographics, ethical considerations, and limitations.

Model Card - Toxicity in Text

Model Details

- The TOXICITY classifier provided by Perspective API [32], trained to predict the likelihood that a comment will be perceived as toxic.
- Convolutional Neural Network.
- Developed by Jigsaw in 2017.

Intended Use

- Intended to be used for a wide range of use cases such as supporting human moderation and providing feedback to comment authors.
- Not intended for fully automated moderation.
- Not intended to make judgments about specific individuals.

Factors

- Identity terms referencing frequently attacked groups, focusing on sexual orientation, gender identity, and race.

Metrics

- Pinned AUC, as presented in [11], which measures threshold-agnostic separability of toxic and non-toxic comments for each group, within the context of a background distribution of other groups.

Ethical Considerations

- Following [31], the Perspective API uses a set of values to guide their work. These values are Community, Transparency, Inclusivity, Privacy, and Topic-neutrality. Because of privacy considerations, the model does not take into account user history when making judgments about toxicity.

Training Data

- Proprietary from Perspective API. Following details in [11] and [32], this includes comments from a online forums such as Wikipedia and New York Times, with crowdsourced labels of whether the comment is “toxic”.
- “Toxic” is defined as “a rude, disrespectful, or unreasonable comment that is likely to make you leave a discussion.”

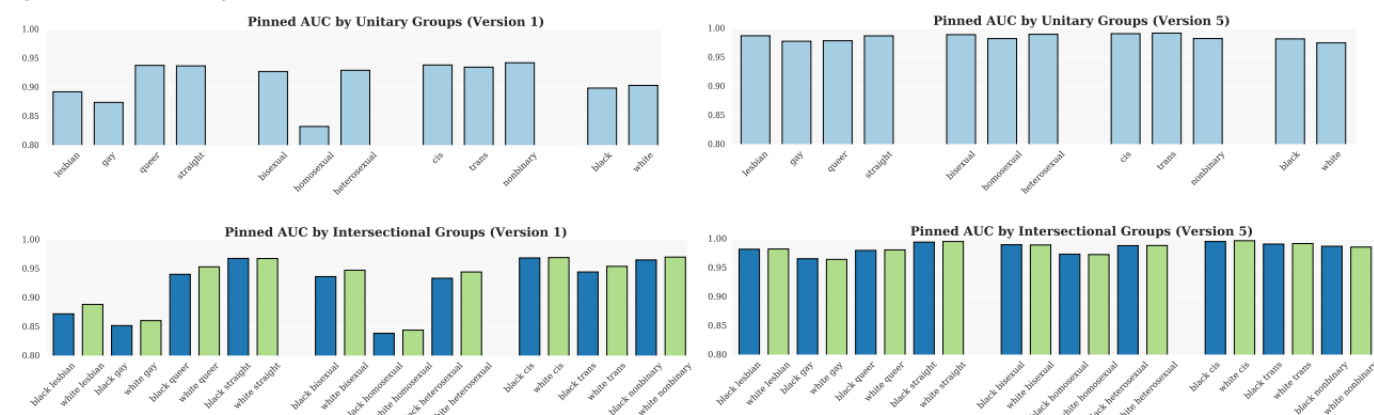
Evaluation Data

- A synthetic test set generated using a template-based approach, as suggested in [11], where identity terms are swapped into a variety of template sentences.
- Synthetic data is valuable here because [11] shows that real data often has disproportionate amounts of toxicity directed at specific groups. Synthetic data ensures that we evaluate on data that represents both toxic and non-toxic statements referencing a variety of groups.

Caveats and Recommendations

- Synthetic test data covers only a small set of very specific comments. While these are designed to be representative of common use cases and concerns, it is not comprehensive.

Quantitative Analyses



Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect

Right after the requirements analysis, programmers have a tendency to jump right into implementations, but this risks building systems that do not actually meet the users' needs, that do not achieve the desired qualities, that do not meet the business goals, and that are hard to change when needed.

Software architecture is the part of the design process that focuses on those early decisions that are most important for achieving the quality requirements of a system (e.g., scalability, flexibility).

Architectural design decisions are fundamental and hard to change later without fully redesigning the system. Yet, those decisions are often difficult to get right in the first place, requiring a good understanding of requirements, of consequences of different designs, and of the trade-offs involved.

The fundamental insight of software architecture is to seriously consider a system's quality requirements early on and design the system to support those qualities.

Quality Requirements

Architecture / Design

Implementation

Architecture is the binding element that guides the implementation to meet the (quality) requirements of the system.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Common Architectural Challenges and System Structures

3 Service-oriented architectures and microservices. A system is organized into multiple self-contained services (processes) that call other services through remote procedure calls. The services are not necessarily organized into layers, and typically each service represents a cohesive piece of functionality and is responsible for its own data storage.

So, to summarize:

Services get registered in the discovery tool, i.e., in tools like Eureka, Consul, or Kubernetes.

Service discovery is decentralized: this tool supports replication and distribution, so that it is not a single point of failure like SOA.

```
graph LR
    Client[Client] -- "sends requests" --> APIGateway[API Gateway]
    APIGateway -- "service discovery" --> SDTool[Service Discovery Tool]
    SDTool -- "finds best service instance" --> Service[Service]
    APIGateway -- "forward client's request" --> Service
    Service -- "response" --> APIGateway
    APIGateway -- "response" --> Client
```

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Quality Attributes and Their Relation to Software Architectures

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Toward Deployment

We can visualize the discussion so far through this chart.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Model Inference in a System

Model inference is an important component of an ML-enabled system, but it needs to be integrated with other components of a system, including non-ML components that request and use the predictions. It is worth deliberately structuring the system.

Designers typically strive to separate concerns. When it comes to machine learning, it is common to clearly separate models from business logic within the system.

Such separation, if done well, makes it easier to encapsulate change and evolve key parts of the system independently.

Similarly, the pipeline infrastructure for training a model and the infrastructure for collecting telemetry and training data can, and usually should, be separated from the non-ML parts of the system to isolate ML-related code and infrastructure and evolve it independently.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Thinking like an Architect: Documenting Model Inference Interfaces

An increasing alternative is called **model card**.

It is brief (one- to two-page) description of models, including intended use, training and evaluation data, considered demographic factors, evaluation results broken down by demographics, ethical considerations, and limitations.

Software Engineering for Artificial Intelligence

Software Architectures and Deployment of AI-enabled Systems

